Assignment 8
DATA 260

Name: Kazi Samina Maraj Mumu

Email: Kazisaminamaraj.mumu@sjsu.edu

Date: 11/01/2026

GitHub repository: https://github.com/SaminaMaraj/data260-4098 (tag hw8)

Personal Configuration and Project Information

SID4	PORT_BASE	PREFIX	SEED	VERIFY_SEED	DOMAIN_ID
4098	8498	s4098	4098	264098	2

Item	Value
Domain	Municipal transit incidents (primary entity: Incident, so CREATE_INCIDENT, GET_INCIDENT and incident_id)
Hardware	Apple MacBook Pro, M5 chip, 24 GB RAM
Local model	qwen3:8b via Ollama (Part 2 agents and the Part 3 consumer-death runs)
Kafka	apache/kafka:3.9.2, one KRaft broker in Docker; Python client aiokafka 0.12.0; Python 3.12
Tagged commit (hw8)	22419af79e3eaa2b29fc75ed16e8d251364f1c21 (the commit the hw8 tag points to)
Verified code commit	d1c53f9b938f14d365e9287de1b0d2cda0dd4ffb (code + all results; make verify-hw08 ran on this commit, 18/18). The later commits up to the hw8 tag change only files in reports/hw08/; the src, code and tests trees in the tag are identical to the ones recorded in verification.json.

I kept extending the same transit incident repository from HW1-HW7 and did not make a separate copy of the code. The new code is in shared root-level folders: `src/hw8_kafka/` (Kafka code, worker, API wrapper, agent loop), `code/hw8/` (Planner, Writer, Reviewer, send_question.py and the Part 3 test runner), `homework_demo.py` and `docker-compose.hw8.yml`. All evidence is in `reports/hw08/` (RUN_LOG.txt, raw/, METRICS.md, AI_USE.md, verification.json).

Part 1 - Backend Service Request/Response over Kafka

I. Setup: topics and containers

Kafka runs as one broker in Docker. I turned off automatic topic creation in the broker, so every topic has to be created on purpose with the right name. I created all seven PREFIX topics with the Kafka CLI (code/hw8/create_topics.sh). For Part 1 only two of them are used: `s4098_entity_requests` and `s4098_entity_responses`.

$ create_topics.sh code/hw8/create_topics.sh

```
#!/usr/bin/env bash
# HW8: create the seven s4098 topics with the Kafka CLI inside the broker container.
#   bash code/hw8/create_topics.sh
set -euo pipefail
echo "Kazi Samina Maraj Mumu | create_topics.sh | PREFIX=s4098"
KT="docker exec s4098-kafka /opt/kafka/bin/kafka-topics.sh --bootstrap-server localhost:9092"

create() {  # name partitions
  $KT --create --if-not-exists --topic "$1" --partitions "$2" --replication-factor 1
}

# Part 1 request/response
create s4098_entity_requests 1
create s4098_entity_responses 1
# Part 2 agent pipeline (tasks has 2 partitions for the Part 3 ordering test)
create s4098_inbox 1
create s4098_tasks 2
create s4098_drafts 1
create s4098_final 1
# Part 3 dead-letter topic
create s4098_dead_letter 1

echo
$KT --list | grep '^s4098_'
echo
$KT --describe --topic s4098_tasks
```

Figure 1. Kafka CLI commands I used to create the topics.

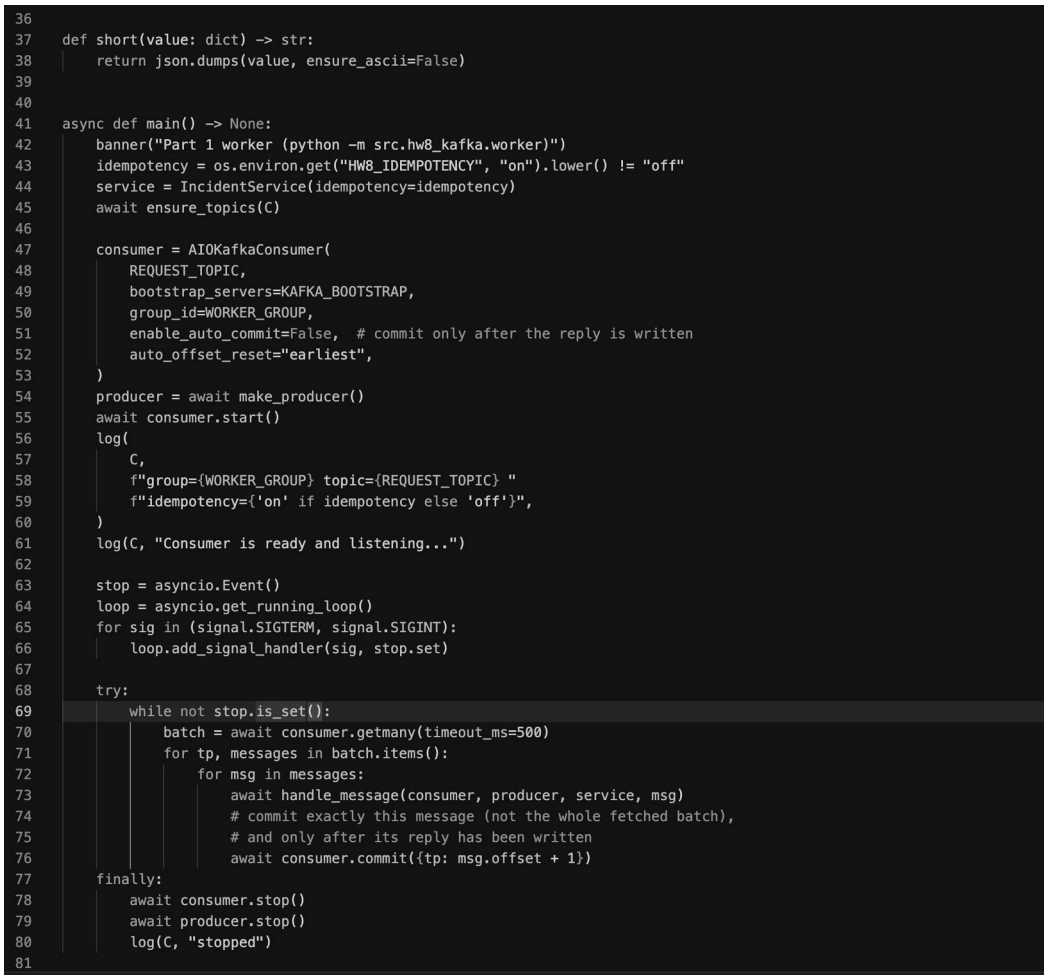
```
 ✔ Container s4098-kafka  Started
((.venv) ) Kazi Samina Maraj Mumu data260-4098 % docker compose -f docker-compose.hw8.yml ps
NAME                IMAGE                COMMAND                  SERVICE       CREATED         STATUS                            PORTS
data260-4098-mongodb   mongo:7              "docker-entrypoint.s…"   mongodb-hw6   2 weeks ago     Up 7 days                         0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
s4098-kafka         apache/kafka:3.9.2   "/__cacert_entrypoin…"   kafka         4 seconds ago   Up 3 seconds (health: starting)   0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
((.venv) ) Kazi Samina Maraj Mumu data260-4098 % docker compose -f docker-compose.hw8.yml ps
NAME                IMAGE                COMMAND                  SERVICE       CREATED          STATUS                    PORTS
data260-4098-mongodb   mongo:7              "docker-entrypoint.s…"   mongodb-hw6   2 weeks ago      Up 7 days                 0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
s4098-kafka         apache/kafka:3.9.2   "/__cacert_entrypoin…"   kafka         23 seconds ago   Up 22 seconds (healthy)   0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
((.venv) ) Kazi Samina Maraj Mumu data260-4098 % bash code/hw8/create_topics.sh
Kazi Samina Maraj Mumu | create_topics.sh | PREFIX=s4098
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
Created topic s4098_entity_requests.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
Created topic s4098_entity_responses.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
Created topic s4098_inbox.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
Created topic s4098_tasks.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
Created topic s4098_drafts.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
Created topic s4098_final.
WARNING: Due to limitations in metric names, topics with a period ('.') or underscore ('_') could collide. To avoid issues it is best to use either, but not both.
Created topic s4098_dead_letter.

s4098_dead_letter
s4098_drafts
s4098_entity_requests
s4098_entity_responses
s4098_final
s4098_inbox
s4098_tasks

Topic: s4098_tasks      TopicId: Q5V1iwizS-G0rKFPk12J4Q PartitionCount: 2       ReplicationFactor: 1    Configs:
        Topic: s4098_tasks      Partition: 0    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
        Topic: s4098_tasks      Partition: 1    Leader: 1       Replicas: 1     Isr: 1  Elr:    LastKnownElr:
((.venv) ) Kazi Samina Maraj Mumu data260-4098 %
```

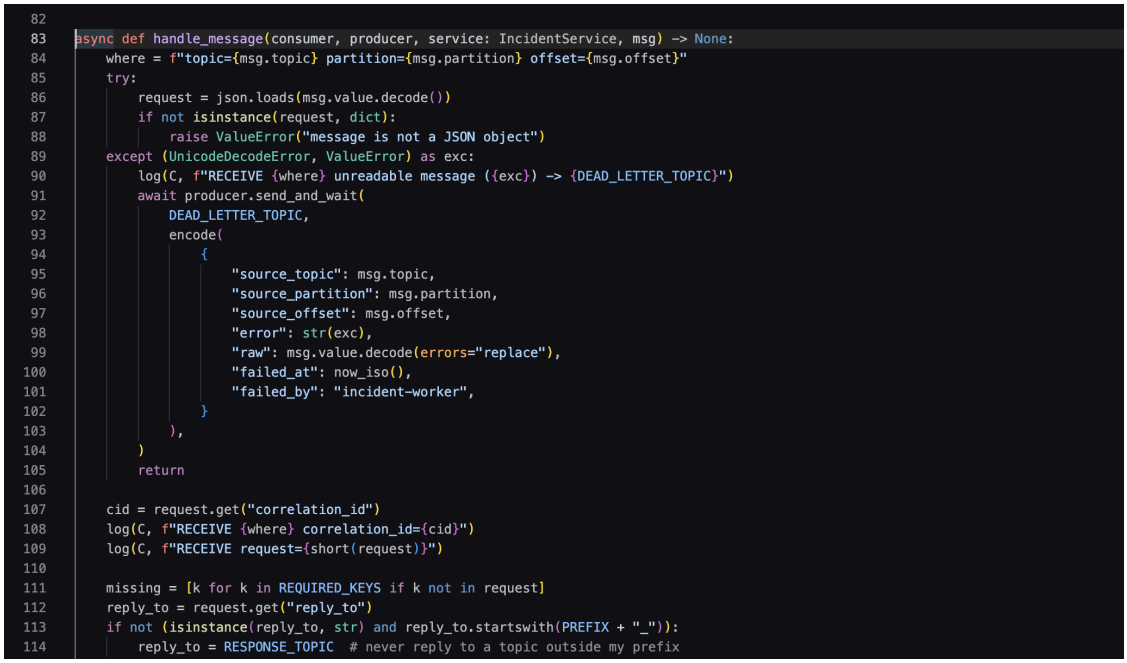
Figure 2. Kafka broker (s4098-kafka) is healthy and the seven s4098_ topics are created with the Kafka CLI. s4098_tasks has 2 partitions for the Part 3 ordering test.

The worker and the FastAPI wrapper are built from the same small image and run as two separate containers next to Kafka. The API is published on my PORT_BASE 8498.



```
36
37  def short(value: dict) -> str:
38      return json.dumps(value, ensure_ascii=False)
39
40
41  async def main() -> None:
42      banner("Part 1 worker (python -m src.hw8_kafka.worker)")
43      idempotency = os.environ.get("HW8_IDEMPOTENCY", "on").lower() != "off"
44      service = IncidentService(idempotency=idempotency)
45      await ensure_topics(C)
46
47      consumer = AIOKafkaConsumer(
48          REQUEST_TOPIC,
49          bootstrap_servers=KAFKA_BOOTSTRAP,
50          group_id=WORKER_GROUP,
51          enable_auto_commit=False,  # commit only after the reply is written
52          auto_offset_reset="earliest",
53      )
54      producer = await make_producer()
55      await consumer.start()
56      log(
57          C,
58          f"group={WORKER_GROUP} topic={REQUEST_TOPIC} "
59          f"idempotency={'on' if idempotency else 'off'}",
60      )
61      log(C, "Consumer is ready and listening...")
62
63      stop = asyncio.Event()
64      loop = asyncio.get_running_loop()
65      for sig in (signal.SIGTERM, signal.SIGINT):
66          loop.add_signal_handler(sig, stop.set)
67
68      try:
69          while not stop.is_set():
70              batch = await consumer.getmany(timeout_ms=500)
71              for tp, messages in batch.items():
72                  for msg in messages:
73                      await handle_message(consumer, producer, service, msg)
74                      # commit exactly this message (not the whole fetched batch),
75                      # and only after its reply has been written
76                      await consumer.commit({tp: msg.offset + 1})
77      finally:
78          await consumer.stop()
79          await producer.stop()
80          log(C, "stopped")
81
```

Figure 3. Worker code (src/hw8_kafka/worker.py): consumer on s4098_entity_requests with auto-commit off; each message goes to handle_message() (Figures 4 and 5), then exactly that message's offset is committed after the reply is written.



```
async def handle_message(consumer, producer, service: IncidentService, msg) -> None:
    where = f"topic={msg.topic} partition={msg.partition} offset={msg.offset}"
    try:
        request = json.loads(msg.value.decode())
        if not isinstance(request, dict):
            raise ValueError("message is not a JSON object")
    except (UnicodeDecodeError, ValueError) as exc:
        log(C, f"RECEIVE {where} unreadable message ({exc}) -> {DEAD_LETTER_TOPIC}")
        await producer.send_and_wait(
            DEAD_LETTER_TOPIC,
            encode(
                {
                    "source_topic": msg.topic,
                    "source_partition": msg.partition,
                    "source_offset": msg.offset,
                    "error": str(exc),
                    "raw": msg.value.decode(errors="replace"),
                    "failed_at": now_iso(),
                    "failed_by": "incident-worker",
                }
            ),
        )
        return

    cid = request.get("correlation_id")
    log(C, f"RECEIVE {where} correlation_id={cid}")
    log(C, f"RECEIVE request={short(request)}")

    missing = [k for k in REQUIRED_KEYS if k not in request]
    reply_to = request.get("reply_to")
    if not (isinstance(reply_to, str) and reply_to.startswith(PREFIX + "_")):
        reply_to = RESPONSE_TOPIC  # never reply to a topic outside my prefix
```

Figure 4. Worker code (src/hw8_kafka/worker.py), handle_message() part 1: reads the request, sends an unreadable message to s4098_dead_letter, reads the correlation_id and checks the required keys and reply_to.



worker.py src/hw8_kafka/worker.py

```
 83  async def handle_message(consumer, producer, service: IncidentService, msg) -> None:
112      reply_to = request.get("reply_to")
113      if not (isinstance(reply_to, str) and reply_to.startswith(PREFIX + "_")):
114          reply_to = RESPONSE_TOPIC  # never reply to a topic outside my prefix
115
116      if missing:
117          response = {
118              "correlation_id": cid,
119              "success": False,
120              "error": "Invalid request: missing " + ", ".join(missing),
121          }
122          duplicate = False
123      else:
124          log(C, f"PROCESS operation={request['operation']} correlation_id={cid}")
125          response, duplicate = service.handle(request)
126          if duplicate:
127              log(
128                  C,
129                  f"DUPLICATE correlation_id={cid} already processed -> returning the "
130                  "cached response, no new incident created (store "
131                  f"size={len(service.incidents)})",
132              )
133          else:
134              log(
135                  C,
136                  f"PROCESS done success={response['success']} correlation_id={cid} "
137                  f"(store size={len(service.incidents)})",
138              )
139
140      meta = await producer.send_and_wait(
141          reply_to, encode(response), key=(cid or "").encode()
142      )
143      log(
144          C,
145          f"REPLY sent to topic={meta.topic} partition={meta.partition} "
146          f"offset={meta.offset} "
147          f"correlation_id={cid}",
148      )
149      log(C, f"REPLY response={short(response)}")
150
151
152  if __name__ == "__main__":
153      asyncio.run(main())
154
```

Figure 5. Worker code, handle_message() part 2: service.handle() validates and processes CREATE/GET, then the response is sent to reply_to with the same correlation_id.

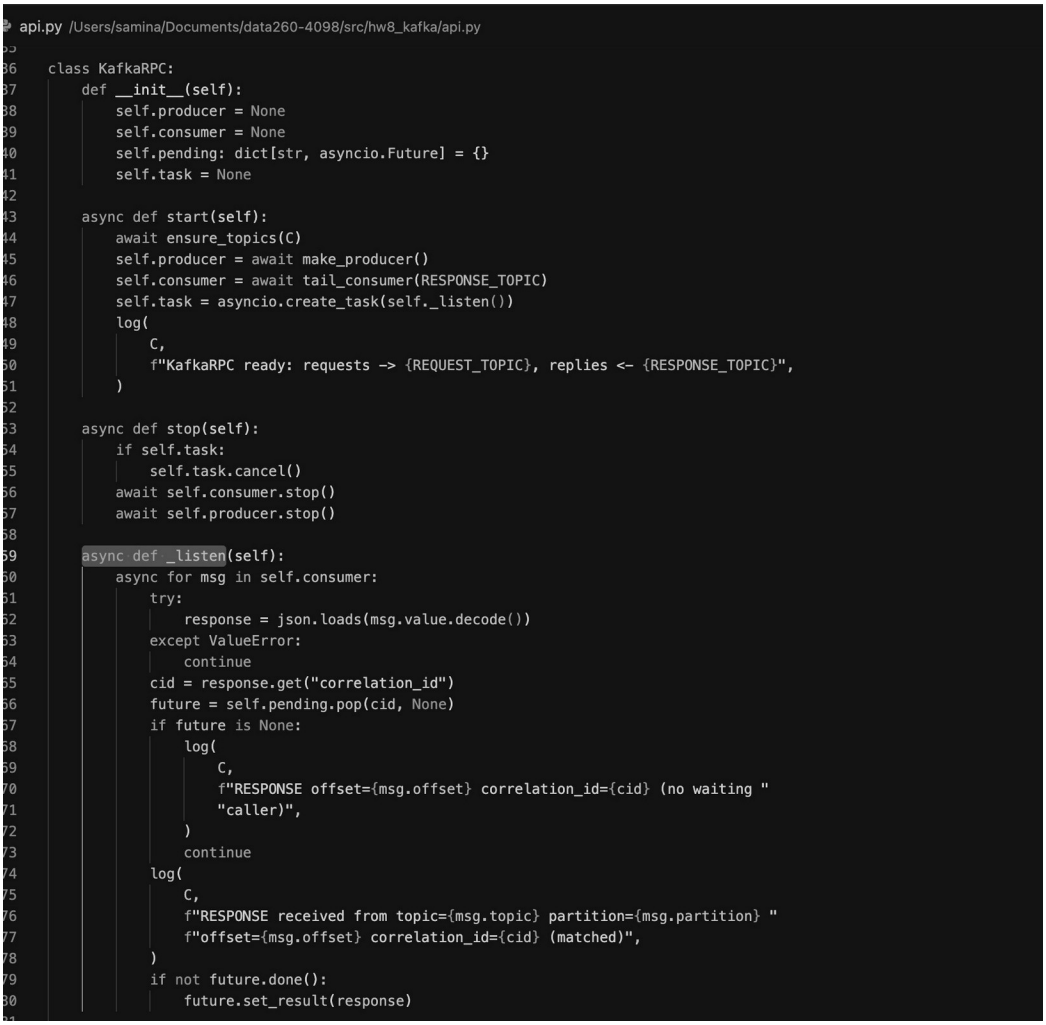
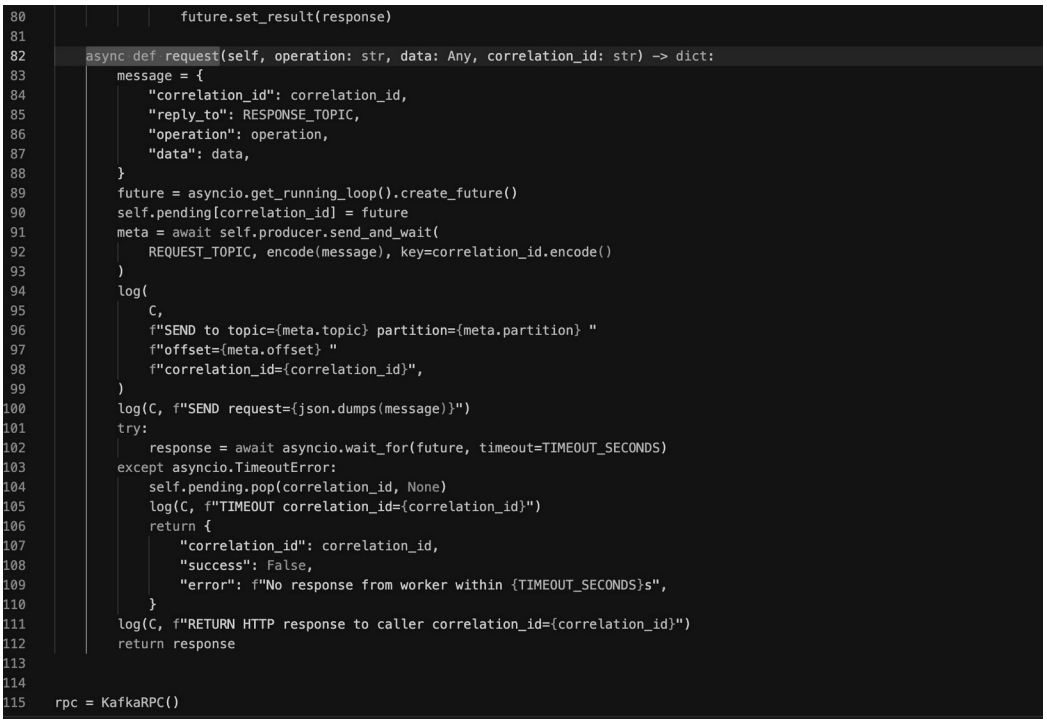


Figure 6. FastAPI wrapper (src/hw8_kafka/api.py): KafkaRPC.start() opens the producer and a listener on s4098_entity_responses; _listen() completes the waiting future whose correlation_id matches.



```
                future.set_result(response)

    async def request(self, operation: str, data: Any, correlation_id: str) -> dict:
        message = {
            "correlation_id": correlation_id,
            "reply_to": RESPONSE_TOPIC,
            "operation": operation,
            "data": data,
        }
        future = asyncio.get_running_loop().create_future()
        self.pending[correlation_id] = future
        meta = await self.producer.send_and_wait(
            REQUEST_TOPIC, encode(message), key=correlation_id.encode()
        )
        log(
            C,
            f"SEND to topic={meta.topic} partition={meta.partition} "
            f"offset={meta.offset} "
            f"correlation_id={correlation_id}",
        )
        log(C, f"SEND request={json.dumps(message)}")
        try:
            response = await asyncio.wait_for(future, timeout=TIMEOUT_SECONDS)
        except asyncio.TimeoutError:
            self.pending.pop(correlation_id, None)
            log(C, f"TIMEOUT correlation_id={correlation_id}")
            return {
                "correlation_id": correlation_id,
                "success": False,
                "error": f"No response from worker within {TIMEOUT_SECONDS}s",
            }
        log(C, f"RETURN HTTP response to caller correlation_id={correlation_id}")
        return response


rpc = KafkaRPC()
```

Figure 7. FastAPI wrapper: request() builds the Kafka message (correlation_id, reply_to, operation, data), saves the future, sends it to s4098_entity_requests and waits for the matching reply (15 s timeout).

api.py src/hw8_kafka/api.py

```python

rpc = KafkaRPC()


@asynccontextmanager
async def lifespan(app: FastAPI):
    banner("Part 1 FastAPI wrapper")
    await rpc.start()
    log(C, f"FastAPI server started at http://127.0.0.1:{PORT_BASE}")
    yield
    await rpc.stop()


app = FastAPI(title=f"HW8 incident Kafka API - {STUDENT_NAME}", lifespan=lifespan)


class IncidentRequest(BaseModel):
    operation: str
    data: Any = Field(default_factory=dict)
    correlation_id: str | None = None


@app.get("/health")
async def health() -> dict:
    return {"status": "ok", "service": "hw8-incident-api", "port": PORT_BASE}


@app.post("/incidents/kafka")
async def incident_operation(body: IncidentRequest) -> dict:
    correlation_id = body.correlation_id or str(uuid.uuid4())
    log(
        C,
        f"HTTP POST /incidents/kafka operation={body.operation} "
        f"correlation_id={correlation_id}",
    )
    return await rpc.request(body.operation, body.data, correlation_id)

```

Figure 8. FastAPI route: POST /incidents/kafka sends the request to Kafka.

topics.py src/hw8_kafka/topics.py

```python

async def make_producer() -> AIOKafkaProducer:
    # acks=all + idempotent producer: a broker retry cannot write a message twice
    producer = AIOKafkaProducer(
        bootstrap_servers=KAFKA_BOOTSTRAP,
        acks="all",
        enable_idempotence=True,
    )
    await producer.start()
    return producer


async def tail_consumer(topic: str) -> AIOKafkaConsumer:
    """Group-less consumer that reads only messages produced from now on.

    It is assigned all partitions directly (no group join, no rebalance wait),
    so a reply cannot be missed between 'send request' and 'start listening'.
    """
    consumer = AIOKafkaConsumer(
        bootstrap_servers=KAFKA_BOOTSTRAP,
        group_id=None,
        enable_auto_commit=False,
    )
    await consumer.start()
    partitions = consumer.partitions_for_topic(topic)
    if not partitions:
        await consumer._client.force_metadata_update()  # noqa: SLF001
        partitions = consumer.partitions_for_topic(topic) or {0}
    tps = [TopicPartition(topic, p) for p in sorted(partitions)]
    consumer.assign(tps)
    await consumer.seek_to_end(*tps)
    for tp in tps:  # resolve the end position now, before anything is sent
        await consumer.position(tp)
    return consumer


if __name__ == "__main__":
    print(asyncio.run(ensure_topics()))

```

Figure 9. Kafka client (src/hw8_kafka/topics.py): make_producer() creates the aiokafka producer, and tail_consumer() creates the consumer the FastAPI wrapper uses to listen on s4098_entity_responses.

```
 => => unpacking to docker.io/library/s4098-hw8-app:latest
[+] up 4/4ing provenance for metadata file
 ✓ Image s4098-hw8-app             Built
 ✓ Container s4098-kafka           Healthy
 ✓ Container s4098-incident-worker Started
 ✓ Container s4098-incident-api    Started
((.venv) ) Kazi Samina Maraj Mumu data260-4098 % docker ps
CONTAINER ID   IMAGE               COMMAND                  CREATED          STATUS                    PORTS                                           NAMES
77ec3a46746e   s4098-hw8-app       "uvicorn src.hw8_kaf…"   35 seconds ago   Up 28 seconds             0.0.0.0:8498->8498/tcp, [::]:8498->8498/tcp     s4098-incident-api
9011df6ae5c9   s4098-hw8-app       "python -u -m src.hw…"   35 seconds ago   Up 28 seconds                                                             s4098-incident-worker
e85123f8a41a   apache/kafka:3.9.2  "/__cacert_entrypoin…"   36 seconds ago   Up 35 seconds (healthy)   0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp     s4098-kafka
c98c02c92ab5   mongo:7             "docker-entrypoint.s…"   2 weeks ago      Up 7 days                 0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp data260-4098-mongodb
a7b35d885c99   mysql:8.4           "docker-entrypoint.s…"   4 weeks ago      Up 7 days                 0.0.0.0:3308->3306/tcp, [::]:3308->3306/tcp     data260-4098-mysql
6e8241a92eb2   mysql:8.4           "docker-entrypoint.s…"   4 weeks ago      Up 7 days                 0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp     handshake-pair15-db
((.venv) ) Kazi Samina Maraj Mumu data260-4098 %
```

Figure 10. docker ps: Kafka broker (s4098-kafka), Part 1 worker (s4098-incident-worker) and FastAPI wrapper (s4098-incident-api, port 8498) running as separate containers.

```
6e8241a92eb2   mysql:8.4           "docker-entrypoint.s…"   4 weeks ago      Up 7 days                 0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp     handshake-pair15-db
((.venv) ) Kazi Samina Maraj Mumu data260-4098 % docker logs s4098-incident-worker
Kazi Samina Maraj Mumu | Part 1 worker (python -m src.hw8_kafka.worker) | PREFIX=s4098 PORT_BASE=8498 KAFKA=kafka:19092
2026-10-30T03:16:37.962Z [WORKER] created topics: s4098_entity_requests, s4098_entity_responses, s4098_inbox, s4098_tasks, s4098_drafts, s4098_final, s4098_dead_letter
Group Coordinator Request failed: [Error 15] GroupCoordinatorNotAvailableError
Marking the coordinator dead (node 1)for group s4098_incident_worker.
2026-10-30T03:16:38.799Z [WORKER] group=s4098_incident_worker topic=s4098_entity_requests idempotency=on
2026-10-30T03:16:38.799Z [WORKER] Consumer is ready and listening...
((.venv) ) Kazi Samina Maraj Mumu data260-4098 %
```

Figure 11. Worker container log: "Consumer is ready and listening..." on s4098_entity_requests in group s4098_incident_worker. The GroupCoordinatorNotAvailable line is a normal one-time retry while the broker finishes starting.

After I recreated the Kafka container for the final run, the worker's ensure_topics() recreated the missing topics with the same names and partition counts, which is the "created topics" line in Figure 11.

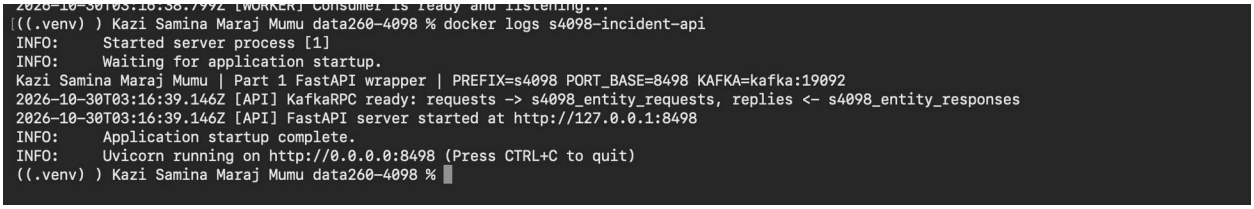


Figure 12. FastAPI container log: KafkaRPC connected (requests -> s4098_entity_requests, replies <- s4098_entity_responses) and "FastAPI server started at http://127.0.0.1:8498".

II. Topics and message schemas

Topic	Who sends -> who reads
s4098_entity_requests	client / FastAPI wrapper -> worker (group s4098_incident_worker)
s4098_entity_responses	worker -> FastAPI wrapper (this is the topic named in reply_to)

Request message (s4098_entity_requests)

```
{ "correlation_id": "<uuid, created once by the client>",
  "reply_to": "s4098_entity_responses",
  "operation": "CREATE_INCIDENT",            or "GET_INCIDENT"
  "data": { "incidentTitle", "routeLine", "submitterEmail", "description",
            "category", "termsAccepted", "passengersAffected" }   or { "incident_id" } }
```

Response message (s4098_entity_responses)

```
{ "correlation_id": "<same value>", "success": true, "operation": "CREATE_INCIDENT",
  "incident_id": "<uuid4>", "message": "Incident created" }
{ "correlation_id": "<same value>", "success": true, "operation": "GET_INCIDENT",
  "incident": { "incident_id": "...", ...all incident fields..., "submissionDate":
"..." } }
{ "correlation_id": "<same value>", "success": false, "error": "Incident not found" }
```

The worker keeps the incidents in a plain Python dictionary. It checks the data with the same rules as my HW4 incident schema: all six required fields must be there and not empty, the email must be valid, the category must be delay, collision, service-suspension or infrastructure-issue, termsAccepted must be true, and unknown fields are rejected. An unknown operation returns "Unknown operation" and an unknown incident_id returns "Incident not found". Every error response still has the original correlation_id.

III. correlation_id flow

- homework_demo.py creates the correlation_id one time (uuid4) and sends it in the HTTP request.
- The FastAPI wrapper keeps the same correlation_id. It only makes one if the caller does not send one (homework_demo.py always sends it). It puts the same value in the Kafka message (also as the message key) and waits on a future saved under that id.
- The worker reads the correlation_id from the request and copies it unchanged into its response, which it sends to the topic in reply_to.
- The wrapper's listener on s4098_entity_responses finds the waiting future with the same correlation_id and returns the response to the client. The client checks that the id is still the same.

IV. Demonstration (homework_demo.py)

homework_demo.py runs the full flow: create a correlation_id, send CREATE_INCIDENT, take the generated incident_id from the response, send GET_INCIDENT with it, and show the retrieved incident. At the end it merges its own log with the worker and API container logs, so one excerpt shows every step in order.

```
68
69  def call(step: str, operation: str, data: dict) -> tuple[str, dict]:
70      cid = str(uuid.uuid4())  # created once, here, by the client
71      clog(f"{step} correlation_id created: {cid}")
72      body = {"correlation_id": cid, "operation": operation, "data": data}
73      clog(
74          f"{step} SEND {operation} correlation_id={cid} via {API} -> Kafka topic "
75          f"{REQUEST_TOPIC} (reply_to={RESPONSE_TOPIC})"
76      )
77      clog(f"{step} request body={json.dumps(body)}")
78      response = post(body)
79      clog(f"{step} RESPONSE received by client: {json.dumps(response)}")
80      same = response.get("correlation_id") == cid
81      clog(f"{step} correlation_id preserved: {same} ({response.get('correlation_id')})")
82      if not same:
83          raise SystemExit("correlation_id changed on the way - this must not happen")
84      return cid, response
```

Figure 13. Client code (homework_demo.py): call() creates the correlation_id once, sends it with the operation and data to the FastAPI wrapper, and checks that the response carries the same correlation_id.

```
148
149 def main() -> int:
150     parser = argparse.ArgumentParser()
151     parser.add_argument(
152         "--log-files", nargs="*", default=[], help="merge these instead of docker logs"
153     )
154     args = parser.parse_args()
155
156     banner("homework_demo.py (Part 1)")
157     since = now_iso()
158
159     print("\n=== Steps 1-7: CREATE_INCIDENT ===")
160     create_cid, created = call("[CREATE]", "CREATE_INCIDENT", INCIDENT)
161     if not created.get("success"):
162         raise SystemExit(f"CREATE failed: {created}")
163     incident_id = created["incident_id"]
164     clog(
165         f"[CREATE] extracted generated incident_id={incident_id} "
166         f"(correlation_id={create_cid})"
167     )
168
169     print("\n=== Steps 8-9: GET_INCIDENT ===")
170     get_cid, fetched = call("[GET]", "GET_INCIDENT", {"incident_id": incident_id})
171
172     print("\n=== Step 10: retrieved incident ===")
173     print(json.dumps(fetched.get("incident"), indent=2))
174     ok = (
175         fetched.get("success") is True
176         and fetched["incident"]["incident_id"] == incident_id
177         and all(
178             fetched["incident"][k] == v
179             for k, v in INCIDENT.items()
180             if k != "description"
181         )
182     )
183     clog(f"[GET] round trip matches the created incident: {ok}")
184     clog(
185         f"[DONE] correlation_id preserved for both operations: "
186         f"CREATE={create_cid} GET={get_cid}"
187     )
188
```

Figure 14. homework_demo.py, main() part 1: sends CREATE_INCIDENT, extracts the generated incident_id, sends GET_INCIDENT with it, shows the retrieved incident and confirms the correlation_id.

```
    )

    print("\n=== Full sequence (client + API + worker logs, ordered by time) ===")
    sequence = merged_sequence([create_cid, get_cid], since, args.log_files)
    for line in sequence:
        print(line)

    RAW.mkdir(parents=True, exist_ok=True)
    (RAW / "part1_sequence.log").write_text("\n".join(sequence) + "\n")
    (RAW / "part1_demo.json").write_text(
        json.dumps(
            {
                "create": {
                    "correlation_id": create_cid,
                    "request": INCIDENT,
                    "response": created,
                },
                "get": {"correlation_id": get_cid, "response": fetched},
                "round_trip_ok": ok,
            },
            indent=2,
        )
    )
    log(C, f"saved {RAW / 'part1_sequence.log'} and part1_demo.json")
    return 0 if ok else 1


if __name__ == "__main__":
    raise SystemExit(main())

```

Figure 15. homework_demo.py, main() part 2: prints the merged client, API and worker sequence and saves part1_sequence.log and part1_demo.json.

My first run of the demo returned HTTP 404. The reason was that my old HW7 API was still running on my Mac on the same port 8498 and answered before Docker. I found the process with `lsof -iTCP:8498`, stopped it, and after that the demo worked. The screenshots below are from the final run.

```
=== Steps 1–7: CREATE_INCIDENT ===
2026-10-30T03:30:17.141Z [CLIENT] [CREATE] correlation_id created: 18afa51f-be37-4fe7-bfb6-7cf0c78b39dc
2026-10-30T03:30:17.141Z [CLIENT] [CREATE] SEND CREATE_INCIDENT correlation_id=18afa51f-be37-4fe7-bfb6-7cf0c78b39dc via http://127.0.0.1:8498/incidents/kafka -> Kafka topic s4098_entity_requests (reply_to=s4098_entity_responses)
2026-10-30T03:30:17.141Z [CLIENT] [CREATE] request body={"correlation_id": "18afa51f-be37-4fe7-bfb6-7cf0c78b39dc", "operation": "CREATE_INCIDENT", "data": {"incidentTitle": "Bus collision at Diridon Station", "routeLine": "VTA Route 22", "submitterEmail": "kazisaminamaraj.mumu@sjsu.edu", "description": "A Route 22 bus hit a parked car while leaving Diridon Station; no injuries, the bus was taken out of service.", "category": "collision", "termsAccepted": true, "passengersAffected": 18}}
2026-10-30T03:30:17.160Z [CLIENT] [CREATE] RESPONSE received by client: {"correlation_id": "18afa51f-be37-4fe7-bfb6-7cf0c78b39dc", "success": true, "operation": "CREATE_INCIDENT", "incident_id": "2ebed720-7589-4b04-a1d1-977edbf1eafc", "message": "Incident created"}
2026-10-30T03:30:17.160Z [CLIENT] [CREATE] correlation_id preserved: True (18afa51f-be37-4fe7-bfb6-7cf0c78b39dc)
2026-10-30T03:30:17.160Z [CLIENT] [CREATE] extracted generated incident_id=2ebed720-7589-4b04-a1d1-977edbf1eafc (correlation_id=18afa51f-be37-4fe7-bfb6-7cf0c78b39dc)
((.venv) ) Kazi Samina Maraj Mumu data260-4098 %
```

Figure 16. homework_demo.py, steps 1-7: the client creates correlation_id 18afa51f..., sends CREATE_INCIDENT with the incident data through FastAPI to s4098_entity_requests, receives success=true with the generated incident_id 2ebed720..., and the correlation_id is preserved.

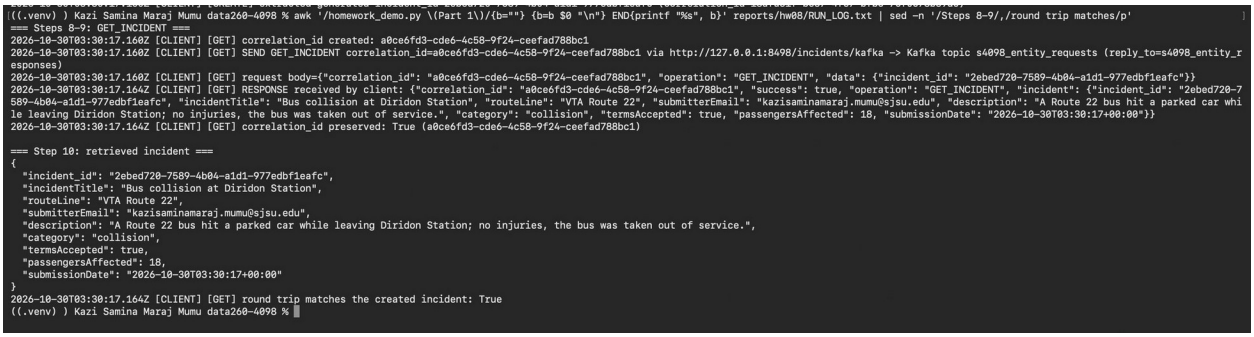


Figure 17. homework_demo.py, steps 8-10: GET_INCIDENT (correlation_id a0ce6fd3...) with the extracted incident_id 2ebed720... returns the stored incident with every field; correlation_id preserved and the round trip matches.



Figure 18. Full message flow (client + API + worker logs). CREATE_INCIDENT (18afa51f...): client -> API sends to s4098_entity_requests (offset 2) -> worker receives and processes -> reply to s4098_entity_responses (offset 2) -> API matches -> client gets incident_id 2ebed720.... Then GET_INCIDENT (a0ce6fd3..., offset 3) returns the same incident. The correlation_id never changes.

src/hw8_kafka/incidents.py - validation (short excerpt)

```
def validate_incident(data) -> str | None:
    missing = [f for f in REQUIRED_FIELDS if f not in data or data[f] in (None, "")]
    if missing:
        return "Missing required field(s): " + ", ".join(missing)
    unknown = sorted(set(data) - ALLOWED_FIELDS)
    if unknown:
        return "Unknown field(s): " + ", ".join(unknown)
    ...                         # then text lengths, email, category, termsAccepted
```

```
Kazi Samina Maraj Mumu data260-4098 % URL=http://127.0.0.1:8498/incidents/kafka; H='Conte
nt-Type: application/json'
curl -s -X POST $URL -H "$H" -d '{"correlation_id":"err-op","operation":"DELETE_INCIDEN
T","data":{}}'; echo
curl -s -X POST $URL -H "$H" -d '{"correlation_id":"err-id","operation":"GET_INCIDENT","d
ata":{"incident_id":"does-not-exist"}}'; echo
curl -s -X POST $URL -H "$H" -d '{"correlation_id":"err-field","operation":"CREATE_INCIDE
NT","data":{"incidentTitle":""}}'; echo
{"correlation_id":"err-op","success":false,"operation":"DELETE_INCIDENT","error":"Unknown
operation"}
{"correlation_id":"err-id","success":false,"operation":"GET_INCIDENT","error":"Incident n
ot found"}
{"correlation_id":"err-field","success":false,"operation":"CREATE_INCIDENT","error":"Miss
ing required field(s): incidentTitle, routeLine, submitterEmail, description, category, t
ermsAccepted"}
Kazi Samina Maraj Mumu data260-4098 %
```

Figure 19. Error responses from my terminal: unknown operation, unknown incident_id and invalid data return success false with the same correlation_id.

Part 2 - Three-Agent Kafka Pipeline

I. Topics and workflow

Sender	Reads topic	Sends to
send_question.py (me)	-	s4098_inbox (a question)
Planner	s4098_inbox	s4098_tasks (a plan with 3-4 steps)
Writer	s4098_tasks (2 partitions)	s4098_drafts (a draft answer)
Reviewer	s4098_drafts	s4098_final (status + answer)

I created these four topics with the Kafka CLI together with the other topics (Figures 1 and 2).

Each agent is its own process in its own terminal with its own consumer group (s4098_planners, s4098_writers, s4098_reviewers). No agent imports or calls another agent; they only share the topic names from `src/hw8_kafka/config.py`. All three use the same Kafka loop in `src/hw8_kafka/agent_runtime.py` and only differ in what they ask qwen3:8b (through my existing `src/model_client.py`). Every agent copies the correlation_id unchanged, uses it as the message key, adds one trace entry (topic, partition, offset, times) and commits its input offset only after its output was sent.

src/hw8_kafka/agents.py - Planner (short excerpt)

```
async def plan(message: dict, ctx) -> dict:
    question = message.get("question")
    if not isinstance(question, str) or not question.strip():
        raise PoisonMessage("inbox message has no question")
    raw = await _chat("You are the Planner ... Return ONLY JSON: steps, focus ...",
                      f"Question: {question}", json_mode=True)
    data = _json_object(raw)
    steps = [str(s).strip() for s in data.get("steps", []) if str(s).strip()][:4]
    if len(steps) < 2:
        raise ValueError(f"planner returned too few steps: {raw[:200]}")
    return {"question": question,
            "plan": {"steps": steps, "focus": str(data.get("focus", ""))}}
```

src/hw8_kafka/agents.py - Writer (short excerpt)

```
async def write(message: dict, ctx) -> dict:
    question, steps = check_task(message)          # a bad plan -> PoisonMessage
    numbered = "\n".join(f"{i}. {s}" for i, s in enumerate(steps, 1))
    draft = await _chat("You are the Writer ... at most 150 words.",
                        f"Question: {question}\nPlan:\n{numbered}", json_mode=False)
    return {"question": question, "plan": message["plan"], "draft": draft.strip()}
```

src/hw8_kafka/agents.py - Reviewer (short excerpt)

```
async def review(message: dict, ctx) -> dict:
    draft = message.get("draft")
    if not isinstance(draft, str) or not draft.strip():
        raise PoisonMessage("draft message has no draft text")
    question = message.get("question", "")
    raw = await _chat("You are the Reviewer ... Return ONLY JSON: approved, feedback ...",
                      f"Question: {question}\nDraft:\n{draft}", json_mode=True)
    data = _json_object(raw)
    approved = data.get("approved") is True
    return {"status": "approved" if approved else "needs_revision", "answer": draft,
            "review": str(data.get("feedback", "")).strip(), "question": question}
```

src/hw8_kafka/agent_runtime.py - what every agent does with one message (short)

```
output = await self.handler(message, self.ctx)          # Planner / Writer / Reviewer
output["correlation_id"] = cid                            # copied unchanged
output["trace"] = message.get("trace", []) + [{"agent": self.name, "in_topic": msg.topic,
                   "partition": msg.partition, "offset": msg.offset, ...}]
await self.producer.send_and_wait(self.out_topic, encode(output), key=cid.encode())
await self.commit(msg)                                    # input offset committed last
```

writer.py code/hw8/writer.py

```python
"""HW8 Part 2 Writer: s4098_tasks (plan) -> s4098_drafts (draft answer).

    python writer.py         (from code/hw8, or: python code/hw8/writer.py)

Run with --help for the Part 3 options (commit mode, poison policy, ...).
"""

import asyncio
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[2]))

from src.hw8_kafka.agent_runtime import Agent, agent_args  # noqa: E402
from src.hw8_kafka.agents import write  # noqa: E402
from src.hw8_kafka.config import DRAFTS_TOPIC, TASKS_TOPIC, WRITER_GROUP  # noqa: E402

if __name__ == "__main__":
    args = agent_args("WRITER", WRITER_GROUP)
    agent = Agent(
        name="WRITER",
        in_topic=TASKS_TOPIC,
        out_topic=DRAFTS_TOPIC,
        handler=write,
        args=args,
    )
    asyncio.run(agent.run())

```

Figure 20. writer.py starts the Writer. planner.py and reviewer.py work the same way.

II. One complete message journey

I started the Planner, Writer and Reviewer in three terminals and then sent one question with send_question.py: "A light-rail train lost power and stopped between two stations during the evening rush. What should the transit agency do in the first 30 minutes?"

```
}
2026-10-30T03:30:17.164Z [CLIENT] [GET] round trip matches the created incident: True
((.venv) ) Kazi Samina Maraj Mumu data260-4098 % code homework_demo.py
((.venv) ) Kazi Samina Maraj Mumu data260-4098 % code src/hw8_kafka/worker.py
((.venv) ) Kazi Samina Maraj Mumu data260-4098 % code src/hw8_kafka/api.py
 ((.venv) ) Kazi Samina Maraj Mumu data260-4098 % PROMPT='Kazi Samina Maraj Mumu %1~ %# '
cd /Users/samina/Documents/data260-4098/code/hw8
source ../../.venv/bin/activate
python planner.py 2>&1 | tee ../../reports/hw08/raw/logs/part2_planner.log
Kazi Samina Maraj Mumu | PLANNER agent instance=planner-1 group=s4098_planners | PREFIX=s4098 PORT_BASE=8498 KAFKA=localhost:9092
2026-10-30T03:40:05.798Z [PLANNER:planner-1] ready: s4098_inbox -> s4098_tasks | commit=after-success poison-policy=dlq max-attempts=3 idempotency=on llm=ollama experiment=None
2026-10-30T03:40:05.801Z [PLANNER:planner-1] assigned s4098_inbox[0] start offset=0
2026-10-30T03:40:05.802Z [PLANNER:planner-1] ASSIGNMENT partitions=[0]
2026-10-30T03:42:13.883Z [PLANNER:planner-1] RECEIVE from s4098_inbox partition=0 offset=0 correlation_id=d2ca2b39-cde9-4288-beb2-46fff1594052
2026-10-30T03:42:13.883Z [PLANNER:planner-1] PROCESS start correlation_id=d2ca2b39-cde9-4288-beb2-46fff1594052 attempt=1
2026-10-30T03:42:19.134Z [PLANNER:planner-1] SEND to s4098_tasks partition=0 offset=0 correlation_id=d2ca2b39-cde9-4288-beb2-46fff1594052
2026-10-30T03:42:19.140Z [PLANNER:planner-1] COMMIT partition=0 offset=1
```

Figure 21. Planner: receives the question from s4098_inbox (offset 0, correlation_id d2ca2b39...), creates a plan with qwen3:8b (about 5 s), sends it to s4098_tasks and only then commits offset 1.

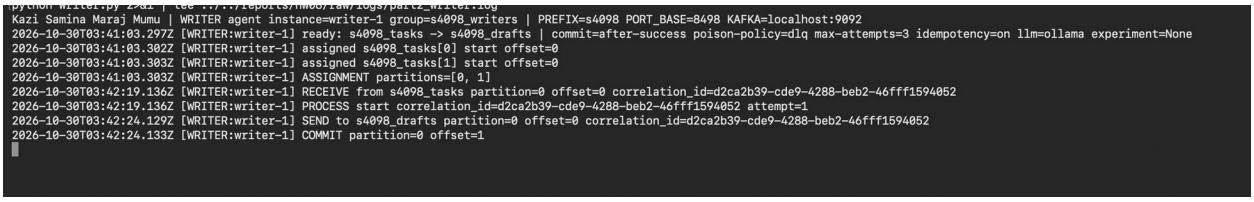


Figure 22. Writer: owns both partitions of s4098_tasks; receives the plan (partition 0, offset 0) with the same correlation_id, writes the draft with qwen3:8b (about 5 s), sends it to s4098_drafts and then commits.

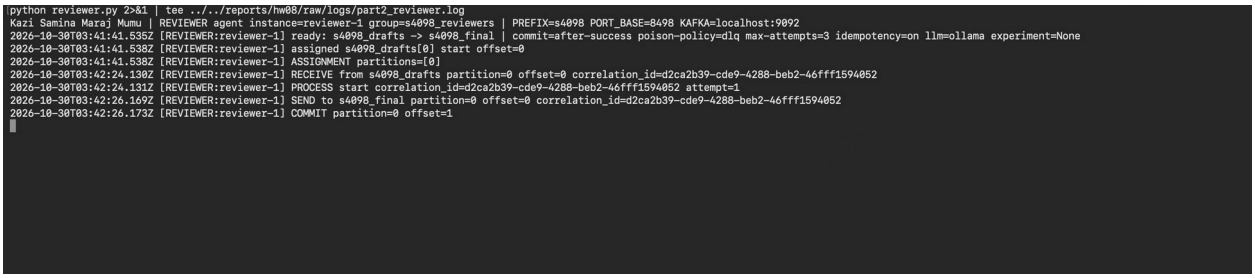


Figure 23. Reviewer: receives the draft from s4098_drafts (offset 0), reviews it with qwen3:8b (about 2 s), sends the approval to s4098_final and then commits.

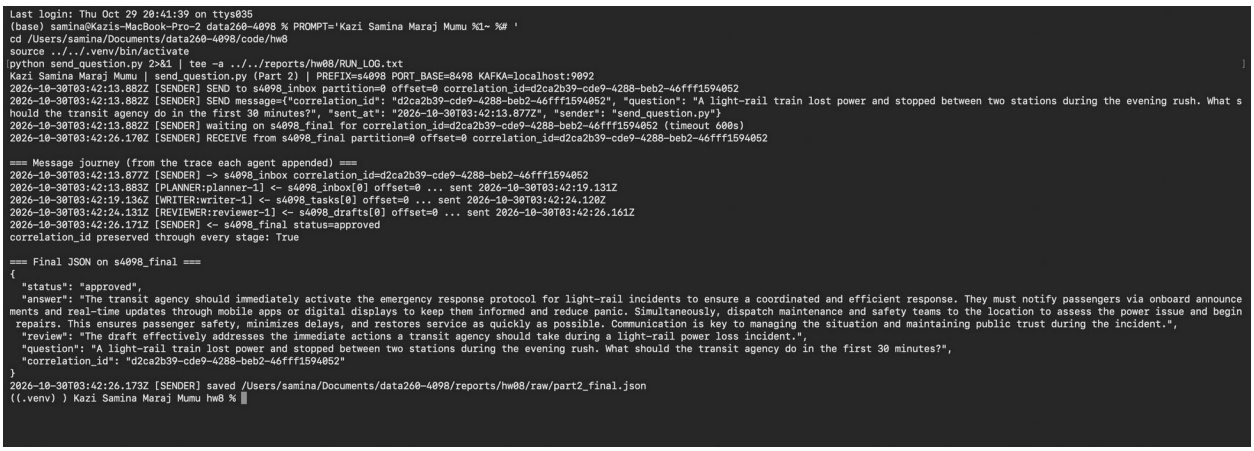


Figure 24. send_question.py: the journey shows every stage with topic, partition, offset and time; the correlation_id is preserved; the final JSON on s4098_final has status "approved" and the answer.

Stage	Read from	Offset	Time in stage
Planner	s4098_inbox[0]	0	5.25 s
Writer	s4098_tasks[0]	0	4.98 s
Reviewer	s4098_drafts[0]	0	2.03 s
Total (send -> final received)			12.3 s

Final message on s4098_final (correlation_id d2ca2b39-cde9-4288-beb2-46fff1594052)


```
{ "status": "approved",
  "answer": "The transit agency should immediately activate the emergency response protocol
     for light-rail incidents ... notify passengers via onboard announcements and real-time
     updates ... dispatch maintenance and safety teams to the location ...",
  "review": "The draft effectively addresses the immediate actions a transit agency should
     take during a light-rail power loss incident.",
  "correlation_id": "d2ca2b39-cde9-4288-beb2-46fff1594052" }
```

III. How Kafka helped my agents coordinate

Kafka was the only connection between the three agents, so each topic worked like a contract. The Planner only had to put a valid plan on s4098_tasks; it did not need to know who reads it or whether the Writer was up. Because messages stay in the topic, an agent that is slow or started later does not lose anything; it just continues from its committed offset. The correlation_id and the trace in every message let me follow one question through all three agents and see exactly how long each stage took (Planner 5.25 s, Writer 4.98 s, Reviewer 2.03 s), without any shared memory between the processes.

Part 3 - Failure Behavior

I ran every failure test with `code/hw8/run_failure_tests.py <test> --mode before|after`. The "before" mode turns one protection off with a flag; the "after" mode is my normal code. For the Writer tests the runner starts its own Writer processes and prints their log lines inline; each of these runs uses its own consumer group and an "experiment" tag on its messages, so test messages never reach my normal Part 2 pipeline. The Part 1 duplicate test sends directly to the worker container and prints the worker's log lines. Every run saves its raw data in reports/hw08/raw/.

I. Duplicate delivery

I replayed the exact same CREATE_INCIDENT message (same bytes, same correlation_id) to the Part 1 worker. Before the fix (worker started with HW8_IDEMPOTENCY=off) every delivery created a new incident. For the fix, the worker remembers the response of every finished CREATE by correlation_id. When the same correlation_id comes again, it sends the saved response back and does not create anything new.

src/hw8_kafka/incidents.py - idempotency

```
def handle(self, request):
    cid = request.get("correlation_id")
    if self.idempotency and cid in self.processed:
        return dict(self.processed[cid]), True   # replay: same answer
    response = {"correlation_id": cid, **self._process(request)}
    if self.idempotency and cid and request.get("operation") == "CREATE_INCIDENT":
        self.processed[cid] = response
    return response, False
```



Figure 25. Duplicate delivery, before the fix: the same CREATE_INCIDENT (correlation_id ef920dc3...) is delivered twice (offsets 4 and 5); the worker creates two different incidents (fb2be043... and 418ef4a7..., store size 1 -> 2).

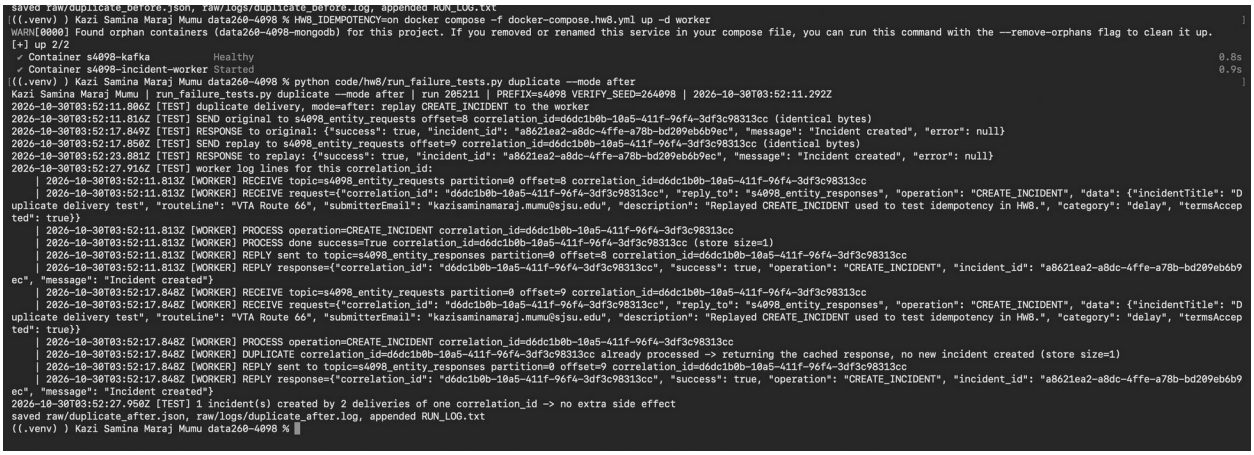


Figure 26. Duplicate delivery, after the fix: the same CREATE_INCIDENT (d6dc1b0b...) is delivered twice (offsets 8 and 9); the second time the worker logs DUPLICATE, returns the same incident_id a8621ea2... and the store size stays 1.

I did the same test on the Writer agent. There the side effect is a second draft.

One limit of my fix: the duplicate protection is an in-memory dictionary (worker) or set (Writer), so it applies only within the same running process. It does not survive a restart and is not shared between different worker instances. A crash after publishing the output but before committing the input offset can still cause a duplicate output after the restart. A fully durable version would need the processed correlation_ids in a shared store (for example a database) or Kafka transactions.

agent_runtime.py src/hw8_kafka/agent_runtime.py

```
119   class Agent:
198       def parse(msg) -> dict:
203           if not isinstance(value, dict):
205               return value
206
207       # ------------------------------------------------------------------ main step
208       async def handle(self, msg) -> None:
209           where = f"partition={msg.partition} offset={msg.offset}"
210           try:
211               message = self.parse(msg)
212           except PoisonMessage:
213               message = None  # unreadable: let the retry policy deal with it
214           if message is not None and message.get("experiment") != self.args.experiment:
215               await self.commit(msg)  # not for this run: skip silently
216               return
217
218           cid = (message or {}).get("correlation_id")
219           if (
220               message is not None
221               and not self.args.no_idempotency
222               and cid in self.completed
223           ):
224               log(
225                   self.tag,
226                   f"DUPLICATE {where} correlation_id={cid} already done -> skipped, no "
227                   "output",
228               )
229               self.record(
230                   event="duplicate_skipped",
231                   correlation_id=cid,
232                   partition=msg.partition,
233                   offset=msg.offset,
234                   at=now_iso(),
235               )
236               await self.commit(msg)
237               return
```

Figure 27. Writer idempotency (agent_runtime.py): if the correlation_id is already done, the Writer logs DUPLICATE, commits the offset and sends nothing. Lines 316-317 in Figure 33 save the finished ids.

```
((.venv) ) Kazi Samina Maraj Mumu data260-4098 % python code/hw8/run_failure_tests.py duplicate-agent --mode before
Kazi Samina Maraj Mumu | run_failure_tests.py duplicate-agent --mode before | run 205308 | PREFIX=s4098 VERIFY_SEED=264098 | 2026-10-30T03:53:08.184Z
2026-10-30T03:53:08.699Z [TEST] duplicate delivery to the Writer, mode=before
    | Kazi Samina Maraj Mumu | WRITER agent instance=writer-dup group=s4098_writers_dup_before_205308 | PREFIX=s4098 PORT_BASE=8498 KAFKA=localhost:9092
    | 2026-10-30T03:53:09.313Z [WRITER:writer-dup] ready: s4098_tasks -> s4098_drafts | commit=after-success poison-policy=dlq max-attempts=3 idempotency=off llm=stub experiment=dup-agent-before-205308
    | 2026-10-30T03:53:09.317Z [WRITER:writer-dup] assigned s4098_tasks[0] start offset=1
    | 2026-10-30T03:53:09.319Z [WRITER:writer-dup] assigned s4098_tasks[1] start offset=0
    | 2026-10-30T03:53:09.319Z [WRITER:writer-dup] ASSIGNMENT partitions=[0, 1]
2026-10-30T03:53:09.427Z [TEST] SEND original task to s4098_tasks[0] offset=1 correlation_id=f5b33b8e-8a3a-40ff-872b-e7e86f8fc1d7
    | 2026-10-30T03:53:09.428Z [WRITER:writer-dup] RECEIVE from s4098_tasks partition=0 offset=1 correlation_id=f5b33b8e-8a3a-40ff-872b-e7e86f8fc1d7
    | 2026-10-30T03:53:09.428Z [WRITER:writer-dup] PROCESS start correlation_id=f5b33b8e-8a3a-40ff-872b-e7e86f8fc1d7 attempt=1
2026-10-30T03:53:09.429Z [TEST] SEND replay task to s4098_tasks[0] offset=2 correlation_id=f5b33b8e-8a3a-40ff-872b-e7e86f8fc1d7
    | 2026-10-30T03:53:09.732Z [WRITER:writer-dup] SEND to s4098_drafts partition=0 offset=1 correlation_id=f5b33b8e-8a3a-40ff-872b-e7e86f8fc1d7
    | 2026-10-30T03:53:09.734Z [WRITER:writer-dup] COMMIT partition=0 offset=2
    | 2026-10-30T03:53:10.034Z [WRITER:writer-dup] RECEIVE from s4098_tasks partition=0 offset=2 correlation_id=f5b33b8e-8a3a-40ff-872b-e7e86f8fc1d7
    | 2026-10-30T03:53:10.034Z [WRITER:writer-dup] PROCESS start correlation_id=f5b33b8e-8a3a-40ff-872b-e7e86f8fc1d7 attempt=1
    | 2026-10-30T03:53:10.337Z [WRITER:writer-dup] SEND to s4098_drafts partition=0 offset=2 correlation_id=f5b33b8e-8a3a-40ff-872b-e7e86f8fc1d7
    | 2026-10-30T03:53:10.337Z [WRITER:writer-dup] COMMIT partition=0 offset=3
    | 2026-10-30T03:53:14.853Z [WRITER:writer-dup] stopped
2026-10-30T03:53:15.080Z [TEST] 2 draft(s) produced for 2 deliveries of one correlation_id
saved raw/duplicate-agent_before.json, raw/logs/duplicate-agent_before.log, appended RUN_LOG.txt
((.venv) ) Kazi Samina Maraj Mumu data260-4098 %
```

Figure 28. Writer, before the fix (idempotency off): the same task (f5b33b8e...) delivered twice gives 2 drafts on s4098_drafts.

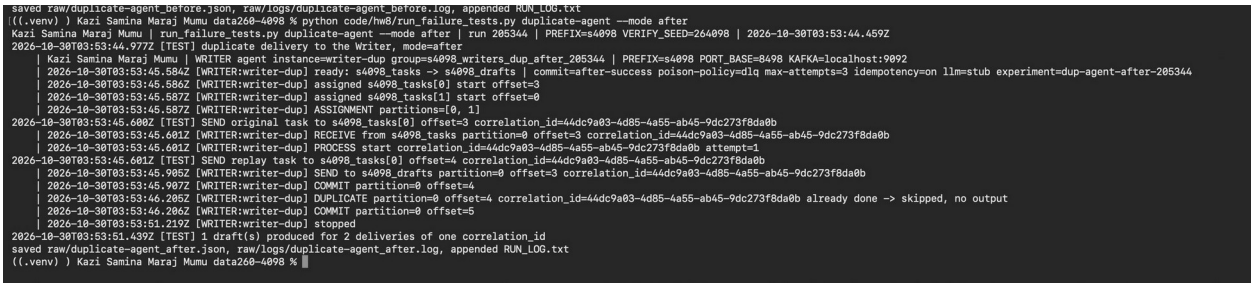


Figure 29. Writer, after the fix: the second delivery of task 44dc9a03... is skipped as DUPLICATE ("already done -> skipped, no output") and only its offset is committed: 1 draft.

II. Consumer death

The runner sends one task, waits until the Writer has started the qwen3:8b call, and then kills the Writer with kill -9 (2.6 s later). After that it restarts the Writer in the same consumer group. Before the fix the Writer used Kafka auto-commit (every 0.5 s), which commits the position of a message as soon as the code gets it, not when the work is done. After the fix auto-commit is off and the offset is committed only after the draft was written to s4098_drafts.

src/hw8_kafka/agent_runtime.py - commit only after success

```
consumer = AIOKafkaConsumer(..., enable_auto_commit=self.auto_commit)   # False normally
...
output = await self.handler(message, self.ctx)
meta = await self.producer.send_and_wait(self.out_topic, encode(output), key=key)
await self.commit(msg)     # commit({TopicPartition(...): msg.offset + 1})
```

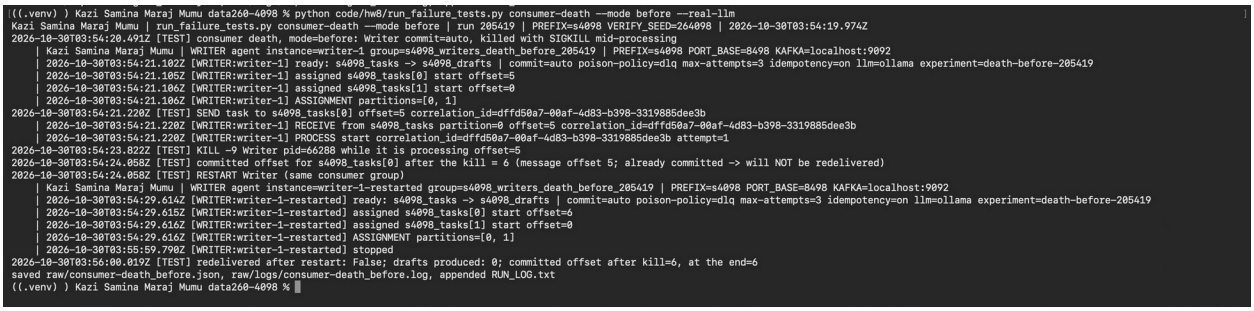


Figure 30. Consumer death, before the fix (auto-commit): the Writer is killed while processing offset 5, but offset 6 was already committed. The restarted Writer starts at offset 6, so the task is never redelivered: 0 drafts, the task is lost.

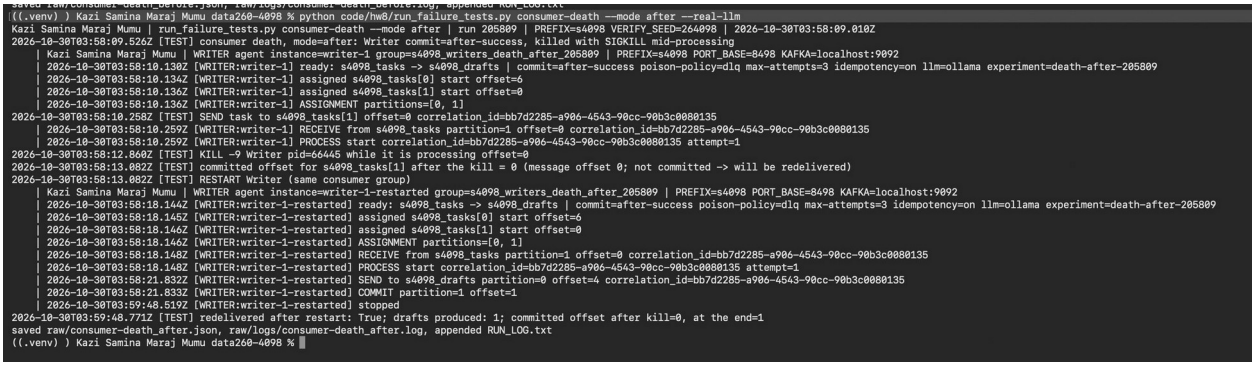


Figure 31. Consumer death, after the fix (commit after success): the Writer is killed while processing partition 1, offset 0; the committed offset is still 0, so Kafka redelivers the same message to the restarted Writer, which finishes it and only then commits offset 1: 1 draft, nothing lost.

I did not only trust my own script here. I also asked Kafka itself with the Kafka CLI, and the committed offset matches.

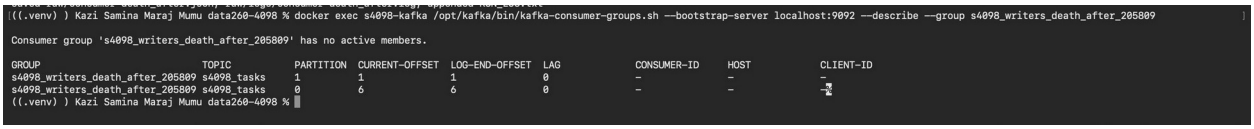


Figure 32. Kafka CLI check (kafka-consumer-groups.sh --describe) for group s4098_writers_death_after_205809: partition 1 has CURRENT-OFFSET 1 = LOG-END-OFFSET 1, lag 0, the same as the runner reported.

III. Poison message

A poison message here is a task whose plan is broken ("<<corrupted plan>>"), so the Writer can never process it. I sent it first and then a good task with the same key, so both are in the same partition. Before the fix the Writer retries forever. After the fix it tries at most 3 times with backoff (0.5 s, then 1 s), then writes the original message, the error, the source offset and the number of attempts to `s4098_dead_letter`, commits the offset and continues.

src/hw8_kafka/agent_runtime.py - bounded retries + dead-letter

```
except Exception as exc:
    if self.args.poison_policy == "block":       # before: no limit
        await asyncio.sleep(self.args.backoff); continue
    if attempt >= self.args.max_attempts:          # after: 3 attempts
        await self.dead_letter(msg, message, error, attempt)   # -> s4098_dead_letter
        await self.commit(msg)                     # the partition is free again
        return
    await asyncio.sleep(self.args.backoff * 2 ** (attempt - 1))
```

agent_runtime.py src/hw8_kafka/agent_runtime.py

```
119   class Agent:
208       async def handle(self, msg) -> None:
305           self.record(
311               seq=message.get("seq"),
312               started_at=started,
313               finished_at=now_iso(),
314               attempts=attempt,
315           )
316           if cid and not self.args.no_idempotency:
317               self.completed.add(cid)
318           await self.commit(msg)  # only now is the input offset committed
319
320       async def dead_letter(self, msg, message, error: str, attempts: int) -> None:
321           entry = {
322               "correlation_id": (message or {}).get("correlation_id"),
323               "experiment": (message or {}).get("experiment"),
324               "source_topic": msg.topic,
325               "source_partition": msg.partition,
326               "source_offset": msg.offset,
327               "failed_by": self.tag,
328               "attempts": attempts,
329               "error": error,
330               "original": msg.value.decode(errors="replace"),
331               "failed_at": now_iso(),
332           }
333           meta = await self.producer.send_and_wait(DEAD_LETTER_TOPIC, encode(entry))
334           log(
335               self.tag,
336               f"DEAD-LETTER {DEAD_LETTER_TOPIC} partition={meta.partition} "
337               f"offset={meta.offset} "
338               f"after {attempts} attempts (source offset={msg.offset}) -> moving on",
339           )
340           self.record(
341               event="dead_lettered",
342               correlation_id=entry["correlation_id"],
343               partition=msg.partition,
344               offset=msg.offset,
345               attempts=attempts,
346               error=error,
347               at=now_iso(),
348           )
349
```

Figure 33. dead_letter() in agent_runtime.py sends the failed message, the error and the attempts to s4098_dead_letter.

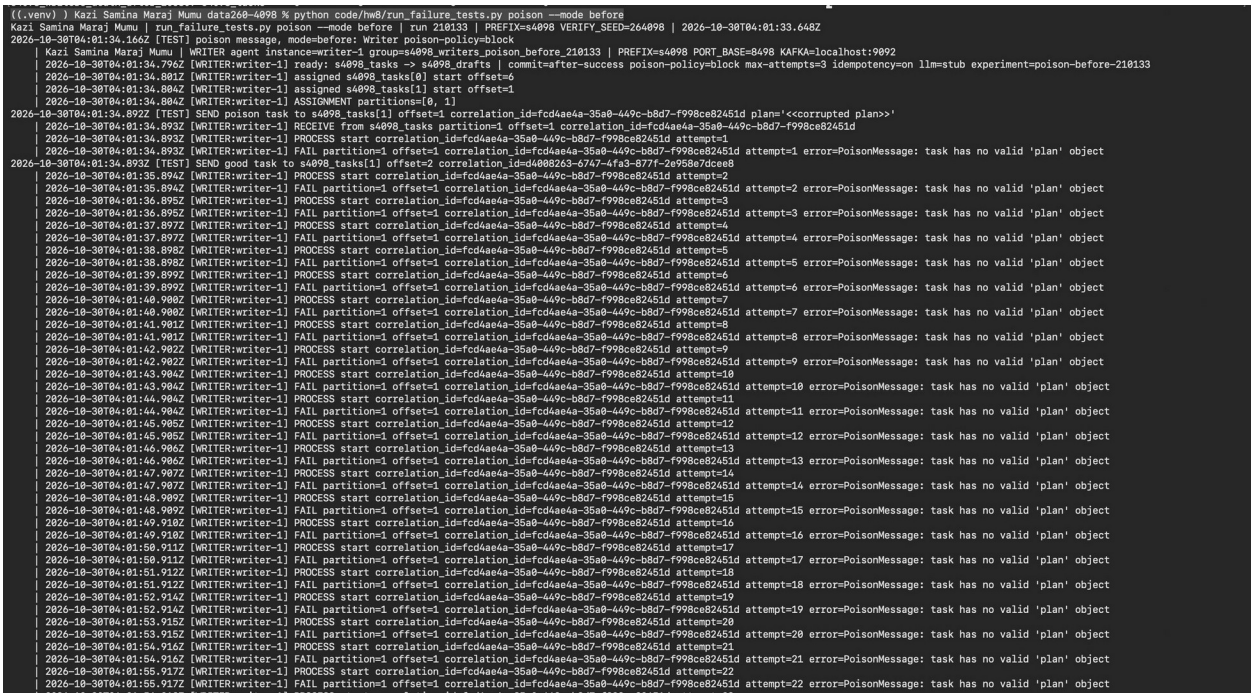


Figure 34. Poison message, before the fix: the corrupted task (partition 1, offset 1) fails and is retried every second without a limit; the good task at offset 2 is stuck behind it.



Figure 35. Poison message, before the fix (end): still failing after 27 attempts. Not dead-lettered and the good message was never processed. (The summary counts 17 attempts because it only counts the 15-second test window; the Writer kept retrying until the test stopped it.)

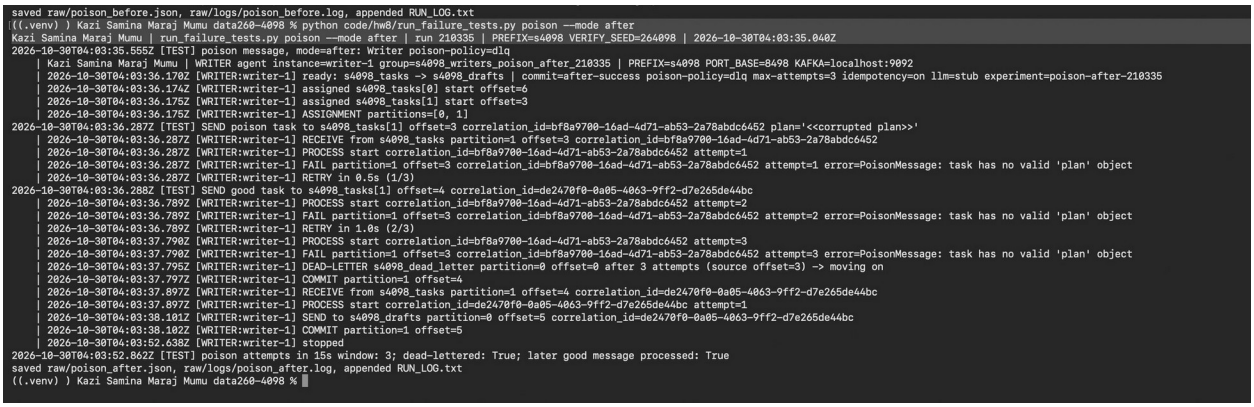


Figure 36. Poison message, after the fix: 3 attempts with backoff 0.5 s -> 1 s, then the task goes to s4098_dead_letter and its offset is committed; the good task at offset 4 is processed right after.

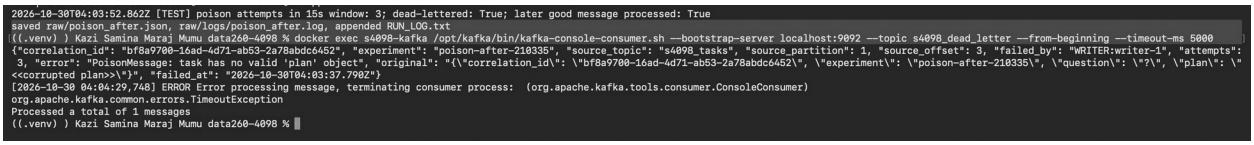


Figure 37. s4098_dead_letter read with the Kafka console consumer: the entry keeps the correlation_id, source topic/partition/offset, who failed it, the 3 attempts, the error and the original message. (The TimeoutException only marks the end of the 5 s read window.)

IV. Ordering

With VERIFY_SEED = 264098 the runner makes 20 related messages: 4 incidents with 5 status updates each (reported, crew_dispatched, crew_on_scene, service_restored, closed, so seq 1-5). The four incident streams were randomly interleaved, while each incident's updates were sent in sequence 1-5, and each message has a random work time. They are sent to s4098_tasks (2 partitions) and read by two Writer instances in the same consumer group, one partition each. In run 1 there is no key. In run 2 the key is the incident_id.

```python
def ordering_messages() -> list[dict]:
    """20 related messages: 4 incidents x 5 status updates, seeded by VERIFY_SEED.

    Per incident the updates are created in order (seq 1..5); the 4 streams
    are interleaved at random, and each message gets a random work time.
    """
    rng = random.Random(VERIFY_SEED)
    incidents = [f"INC-{rng.randint(100000, 999999)}" for _ in range(4)]
    statuses = [
        "reported",
        "crew_dispatched",
        "crew_on_scene",
        "service_restored",
        "closed",
    ]
    next_seq = {i: 1 for i in incidents}
    messages = []
    while len(messages) < 20:
        incident = rng.choice([i for i in incidents if next_seq[i] <= 5])
        seq = next_seq[incident]
        next_seq[incident] += 1
        messages.append(
            {
                "incident_id": incident,
                "seq": seq,
                "status": statuses[seq - 1],
                "work_ms": rng.randint(50, 700),
                "plan": {
                    "steps": [
                        f"Update rider notice: {incident} is {statuses[seq - 1]}"
                    ],
                    "focus": "status update",
                },
            }
        )
    return messages
```

Figure 38. run_failure_tests.py: makes the 20 related messages with VERIFY_SEED.

code/hw8/run_failure_tests.py - partition key

```
key = m["incident_id"].encode() if keyed else None      # run 2: key = incident_id
meta = await producer.send_and_wait(TASKS_TOPIC, encode(value), key=key)
```

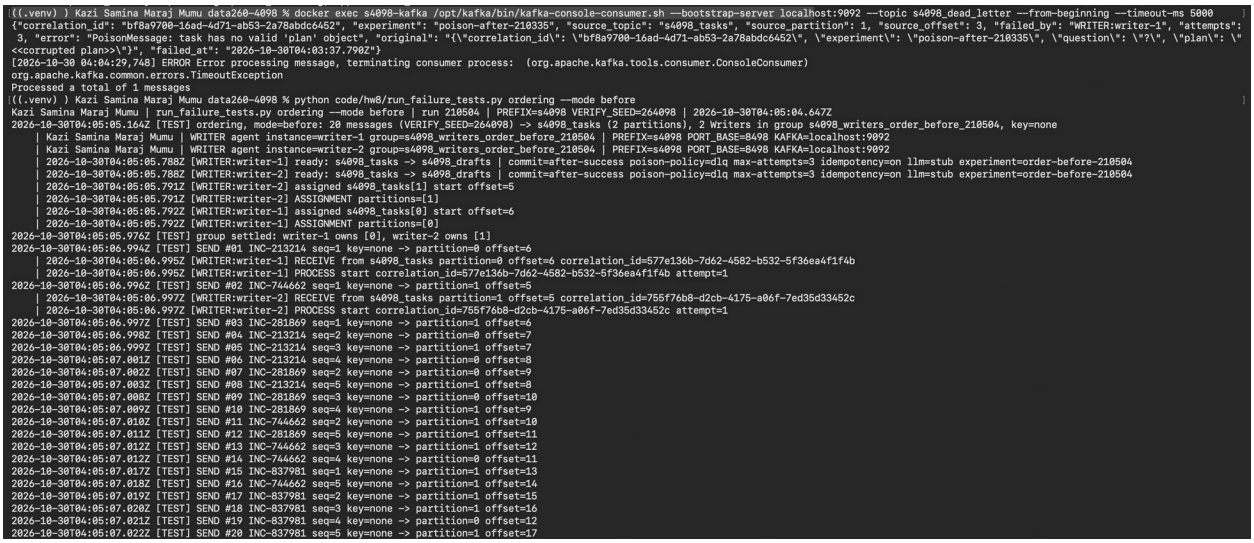


Figure 39. Ordering run 1 (no key): writer-1 owns partition 0 and writer-2 owns partition 1; the 20 VERIFY_SEED messages land in both partitions, so the updates of every incident are split.

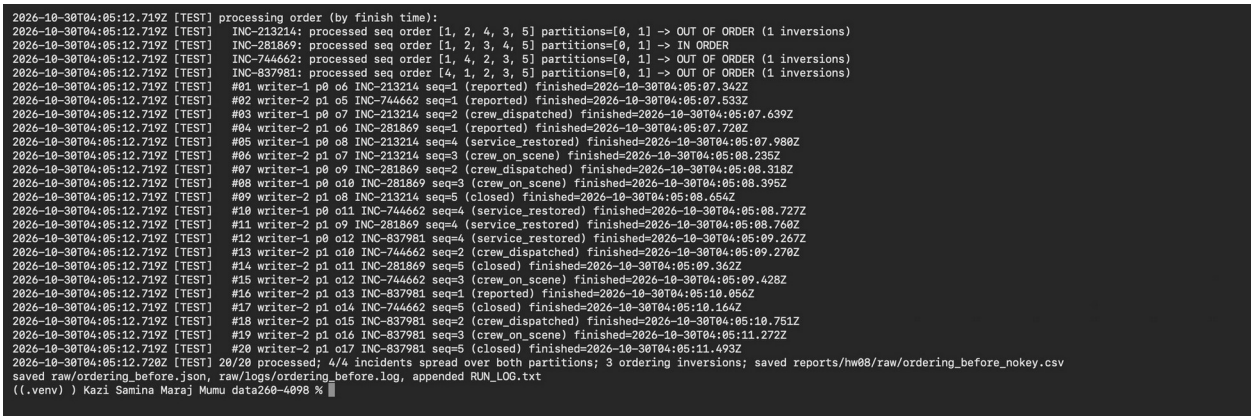


Figure 40. Ordering run 1 result: 3 of 4 incidents processed out of order (INC-213214: 1,2,4,3,5; INC-744662: 1,4,2,3,5; INC-837981: 4,1,2,3,5, so "service_restored" before "reported").

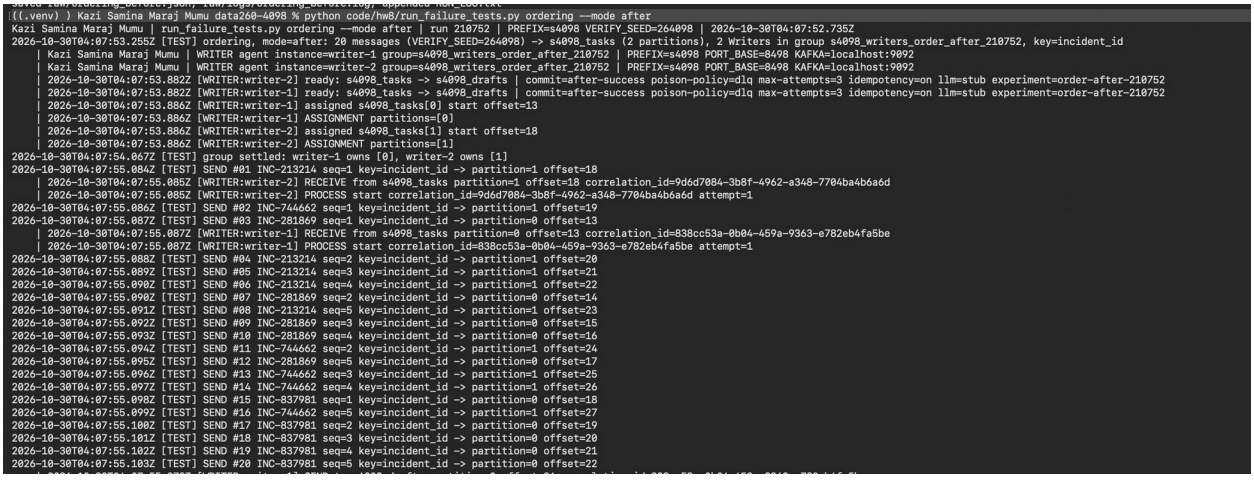


Figure 41. Ordering run 2 (key = incident_id): every update of one incident goes to the same partition (INC-213214 and INC-744662 -> 1; INC-281869 and INC-837981 -> 0).

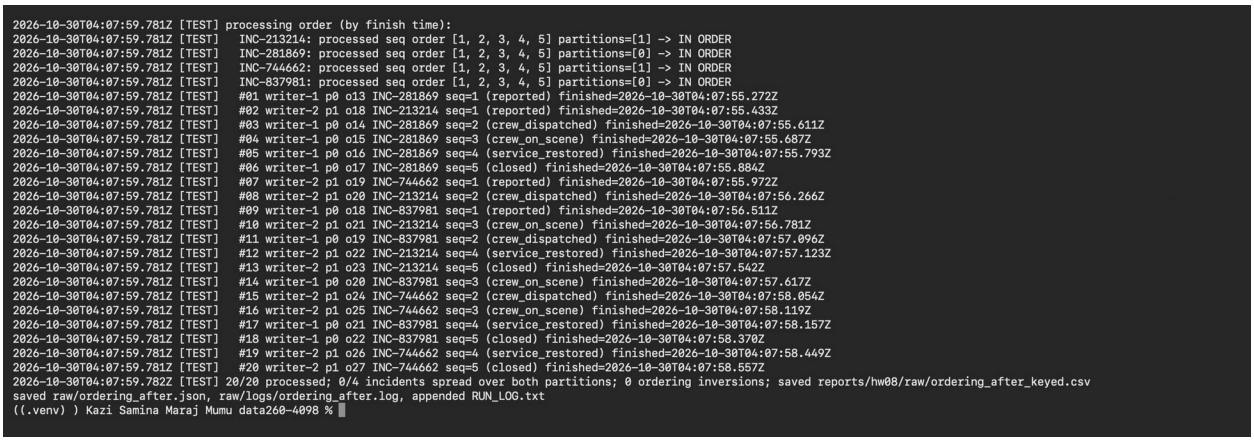


Figure 42. Ordering run 2 result: each incident stays in one partition and is processed by one Writer in offset order, so all 4 incidents are 1,2,3,4,5. The two Writers still work in parallel on different incidents.

Kafka only keeps order inside one partition. With no key, the two updates of one incident can sit in different partitions and two Writers can finish them in any order. With incident_id as the key, all updates of an incident are in one partition and keep their order, and different incidents can still run in parallel.

V. Failure table

Failure scenario	Before fix	After fix
Duplicate delivery (worker)	2 incidents created from 2 deliveries of one correlation_id (Fig. 25)	1 incident; replay gets the same incident_id, no side effect (Fig. 26)
Duplicate delivery (Writer)	2 drafts for 1 task (Fig. 28)	1 draft; replay skipped as DUPLICATE (Fig. 29)
Consumer killed mid-work	Offset already committed (6) -> not redelivered, 0 drafts, task lost (Fig. 30)	Offset not committed (0) -> redelivered, 1 draft, committed after (Fig. 31, 32)
Poison message	Retried forever (27 attempts), not dead-lettered, good message blocked (Fig. 34, 35)	3 attempts -> s4098_dead_letter, good message processed (Fig. 36, 37)
Two partitions/two consumers	No key: 4/4 incidents in both partitions, 3 out of order (Fig. 39, 40)	Key = incident_id: 1 partition per incident, 0 out of order (Fig. 41, 42)

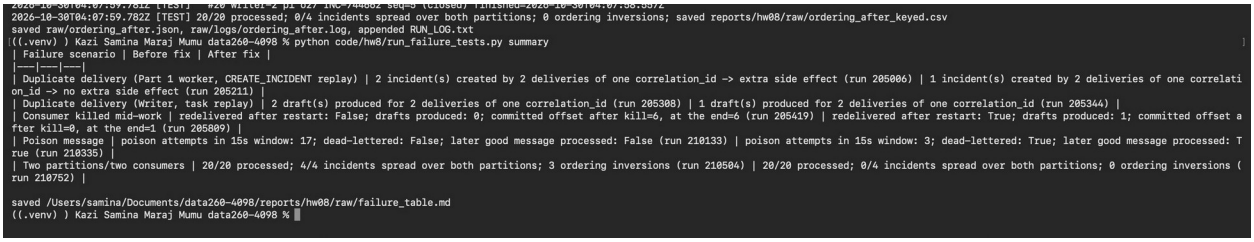


Figure 43. Failure table generated by the runner from reports/hw08/raw/failure_results.json (every row with its run id).

VI. Comparison with the HW2 in-process graph

In HW2 my LangGraph graph ran Planner, Reviewer and Supervisor as function calls in one process. Each step waited for the step before it and they all shared one state object.

Two benefits I got with Kafka:

- **Independent scaling.** In the ordering test I added a second Writer just by starting it in the same consumer group, and Kafka split the two partitions of s4098_tasks between them. In HW2 the only way to scale was to run the whole graph again.
- **Decoupled failure.** When the Writer was killed, the task waited in Kafka and was finished after the restart from the committed offset, and the other agents were not affected. In HW2 an uncaught error in one node, such as a failed model call, ended the whole run and the in-memory state was gone.

Two new responsibilities I now have to handle myself:

- **Duplicate delivery and offsets.** Kafka delivers at least once, so I had to decide when to commit (only after the output is written) and make processing idempotent by correlation_id. In my normal HW2 run, each node was called once; I did not implement message redelivery or distributed offset management.
- **Ordering and bad messages.** Kafka keeps order only inside one partition, so I had to choose a message key (incident_id). And a message that always fails needs a retry limit and a dead-letter topic, otherwise it blocks its partition. In HW2 the order was fixed in the code, and a bad output went back to the Planner until the Supervisor stopped at the turn ceiling.

Verification

`make verify-hw08` is a smoke test on the committed code. It uses the running Docker stack, starts its own short-lived agents with a stub model (so it does not need Ollama), and checks behavior, not wording: the topics and partitions, the API CREATE/GET round trip, error responses with the same correlation_id, a direct Kafka round trip, the idempotent replay, the three-agent pipeline, retry + dead-letter, key -> partition with the VERIFY_SEED messages, and the offline tests. It writes reports/hw08/verification.json with the homework number, SID4, commit hash, model, SEED/VERIFY_SEED and every check. All 18 checks passed on commit d1c53f9.

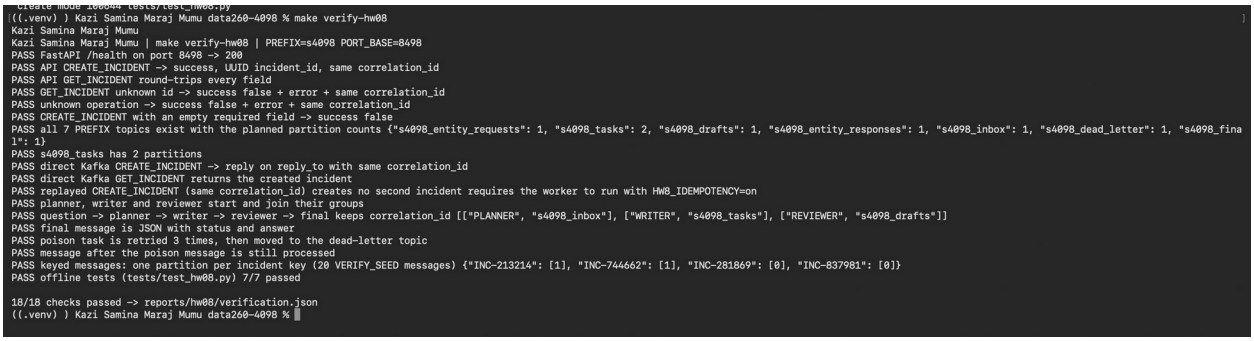


Figure 44. make verify-hw08: 18/18 checks passed, written to reports/hw08/verification.json.

AI Use

1. What did I use an AI assistant for, and what did I do myself?

I used Claude to read the HW8 assignment, the class demos and my repository, and to write the first version of the HW8 code: the Kafka worker and FastAPI wrapper for my Incident entity, homework_demo.py, the docker compose file, the Planner/Writer/Reviewer agents and their shared Kafka loop, the Part 3 test runner, the offline tests, verify_hw08.py and a run guide. Claude also helped me write this report from my screenshots and results. I ran everything myself on my Mac: creating the topics, starting the containers, the demo, the four agent terminals, all Part 3 before/after runs and make verify-hw08. All experiments used the real Kafka broker (apache/kafka:3.9.2). Part 2 and the consumer-death runs used qwen3:8b; the Writer duplicate, poison-message, ordering and verification tests used a stub model, because they test Kafka behavior, not the model (the Part 1 duplicate test uses no model). I took all screenshots, checked the consumer-death result with the Kafka CLI myself, and did the commit, tag and push.

2. One AI-produced output that was wrong, and one thing I verified myself.

Claude's first worker read messages in batches with getmany() and then called consumer.commit() with no arguments after the first message. That commits the position after the whole batch, so messages 2..n of a batch would have been marked as done before they were processed. This is exactly the "offset committed before processing" problem from Part 3 II. A second problem was in the first homework_demo.py: it sorted the merged client/API/worker log only by timestamp, and because several lines have the same millisecond, the API's "response received" line was printed before the worker's "reply sent" line, which looks impossible. I also verified one result myself instead of trusting the test script: for the consumer-death after-run I asked Kafka directly with kafka-consumer-groups.sh --describe for group s4098_writers_death_after_205809, and it showed CURRENT-OFFSET 1 = LOG-END-OFFSET 1 with lag 0 on partition 1, the same value the script reported "at the end".

3. How the problems were found / how I verified.

The batch-commit bug was found while reviewing the commit code against how aiokafka's commit() works without offsets (it commits the current position of every assigned partition). The log-order problem was visible in the first test output of the demo. For my own check I used the Kafka CLI on the real broker (Figure 32) and compared the committed offset with the message offset (0) and with the runner's value (1).

4. What was changed and why it works now.

Every consumer now commits exactly one message, commit({TopicPartition: msg.offset + 1}), and only after its output (response, plan, draft or final) was written with send_and_wait. In my consumer-death after-run the committed offset stayed at 0 after the kill, so the restarted Writer got the same message again and finished it; in the before-run (auto-commit) the offset was already 6 and the task was lost. The demo now orders the merged log by operation and by the stage of the round trip (client send -> API send -> worker receive/process/reply -> API response -> client) and keeps the real timestamps, so the sequence in Figure 18 is in the right order.